

Aula 2 — Entrada e Saída Digital

O latch, o pino, e a distância entre os dois

Microcontroladores — DENE/UFMT

Objetivos da aula

- Descrever o circuito interno de um pino de entrada e saída e explicar o comportamento elétrico que dele decorre.
- Distinguir os três registradores de cada porta — direção, pino e latch — e enunciar a regra de uso que evita a classe de defeitos mais sutil deste periférico.
- Identificar os pinos do PIC18F4550 que não se comportam como entrada e saída genérica e as configurações necessárias para que se comportem.
- Dimensionar corrente em cargas simples, respeitando os limites por pino e por conjunto de pinos.
- Explicar a origem física do ruído de contato e implementar filtragem por espera e por amostragem periódica.

1 O pino por dentro

Um pino de entrada e saída parece trivial — é apenas um fio que assume dois valores. O circuito que há atrás dele, porém, explica quase todo o comportamento inesperado que aparece no laboratório.

Elemento	Função e consequência prática
Par complementar de saída	Dois transistores em série entre a alimentação e o terra, com o pino no ponto médio. Conduzem um de cada vez: o superior leva o pino ao nível alto, o inferior ao nível baixo. Quando ambos estão desligados, o pino fica em alta impedância — é o estado de entrada
Latch de saída	Elemento de memória que guarda o valor escrito pelo programa. É ele que comanda o par de saída, e não o programa diretamente
Buffer de entrada	Converte a tensão do pino em nível lógico. Em alguns pinos é do tipo Schmitt, com histerese; em outros, de limiar simples
Resistor de elevação interno	Disponível apenas em uma das portas, e habilitado em conjunto para toda ela. Dispensa o resistor externo em botões
Diodos de proteção	Conduzem tensões acima da alimentação ou abaixo do terra para os trilhos, protegendo a entrada. Toleram corrente limitada

Tabela 1: Elementos do circuito de um pino e o que cada um implica.

Observação

Duas consequências merecem destaque desde já. A primeira: **o latch e o pino são coisas diferentes**, separadas por um transistor e por tudo o que estiver ligado externamente — e a seção 3 é inteiramente sobre essa distância. A segunda: a histerese do buffer Schmitt reaparece aqui, a mesma ideia da Aula 5, agora protegendo a entrada digital contra oscilações em torno do limiar.

2 Os três registradores de cada porta

Registrador	O que faz	Ao ser lido, devolve
TRISx	Define a direção: bit em 1 para entrada, bit em 0 para saída	A configuração corrente
PORTx	Escrita vai para o latch; leitura vem do pino	A tensão presente no pino , convertida em nível lógico
LATx	Acesso direto ao latch de saída	O valor escrito , independentemente do que ocorre no pino

Tabela 2: Registradores de porta no PIC18.

A mnemônica para a direção causa confusão até se fixar: **1 para entrada porque a letra I de *input* lembra o algarismo 1; 0 para saída porque a letra O de *output* lembra o zero.**

Atenção

O terceiro registrador é a novidade em relação à família anterior, e existe por um motivo específico. Não é conveniência de nomenclatura: é a solução de um defeito real, descrito a seguir.

3 O problema da leitura-modificação-escrita

Alterar um único bit de uma porta parece uma operação atômica quando escrita em C, mas não é. O processador não tem instrução para “ligar o bit 3 do pino”: ele lê o byte inteiro, altera o bit em questão e escreve o byte inteiro de volta.

Derivação

Considere PORTD com todos os pinos configurados como saída e o latch contendo 0b00000011. O pino RD0 aciona um LED com resistor, e o pino RD1 aciona a base de um transistor cuja carga puxa o pino para um valor de tensão intermediário — ou, no laboratório, um pino ligado a um capacitor que ainda não carregou.

A instrução `PORTDbits.RD7 = 1;` produz:

1. **leitura de PORTD** — que devolve o estado elétrico dos **pinos**, não do latch. O pino RD1, carregado, é lido como 0. O byte lido é 0b00000001;
2. **modificação** — o bit 7 é ligado: 0b10000001;
3. **escrita** — o byte inteiro volta ao latch.

O latch, que continha 0b00000011, passa a conter 0b10000001. **O bit 1 foi desligado sem que nenhuma linha do programa pedisse isso.** A operação sobre RD7 corrompeu RD1.

Atenção

Os sintomas desse defeito são característicos: um pino “desliga sozinho”, o defeito aparece só quando certa carga está conectada, e desaparece na simulação — porque o simulador frequentemente devolve o valor do latch na leitura da porta, e não a física do pino. Trocar o código não resolve; trocar o **registrador** resolve.

3.1 A regra

Definição --- regra de acesso às portas

Ler sempre de PORTx. Escrever sempre em LATx. A leitura precisa vir do pino, porque é isso que se quer saber de uma entrada. A escrita precisa ir ao latch, porque é o valor pretendido, e não o valor medido, que deve ser preservado nos demais bits.

Código

```
/* Errado: le o pino, modifica, escreve no latch. */
PORTDbits.RD7 = 1;
PORTD |= 0x80;

/* Certo: le o latch, modifica, escreve no latch. */
LATDbits.LATD7 = 1;
LATD |= 0x80;

/* Certo: leitura de entrada vem do pino. */
if (PORTBbits.RB0 == 0) {
    /* botao pressionado */
}
```

Observação

Na família anterior, sem o registrador de latch, a solução era manter uma cópia da porta em memória — uma variável espelho — modificá-la e escrever o byte inteiro. O latch é exatamente essa variável espelho, implementada em hardware. Reconhecer isso é útil porque a técnica do espelho continua válida em qualquer arquitetura que não ofereça o recurso.

4 Pinos que não são o que parecem

Ao energizar, nem todos os pinos do PIC18F4550 estão prontos para funcionar como entrada e saída digital. Esta seção é a lista de verificação que resolve a maior parte dos “meu programa está certo mas não funciona” da primeira semana.

Situação	Efeito	Providência
Pinos analógicos após o reset	Vários pinos das portas A e E iniciam como entrada analógica e não respondem como digital	Configurar o registrador de controle do conversor para o modo inteiramente digital
Porta B após o reset	Parte dos pinos pode iniciar como analógica, conforme um bit de configuração	Desabilitar essa opção na palavra de configuração
Comparadores ativos	Ocupam pinos da porta A	Desligar o módulo na inicialização
Dois pinos da porta C	São apenas entrada : pertencem ao módulo USB e não possuem transistor de saída	Não usá-los como saída; nenhum ajuste os habilita

Situação	Efeito	Providência
Um pino da porta E	É apenas entrada, compartilhado com a reinicialização externa	Usar somente como entrada
Um pino da porta A	Indisponível quando o oscilador externo está em uso	Considerá-lo ocupado nos modos com cristal

Tabela 3: Pinos com comportamento especial e o que fazer a respeito.

Atenção

A quarta linha não tem solução por software. É comum um estudante configurar a direção como saída, escrever no latch, medir com o multímetro e não encontrar tensão alguma — e concluir que o chip está queimado. **Não há transistor de saída nesses pinos.** Uma leitura da tabela de pinos da folha de dados, antes de fiar, economiza uma tarde.

Código

```
/* Preambulo de inicializacao. Sem estas linhas, boa parte
   dos pinos nao responde como entrada e saida digital. */

ADCON1 = 0x0F;          /* todos os canais como digital */
CMCON   = 0x07;          /* comparadores desligados */

TRISD   = 0x00;          /* porta D inteira como saida */
LATD    = 0x00;          /* estado inicial definido ANTES */
```

Observação

A ordem das duas últimas linhas importa mais do que parece. Definir o latch **antes** de configurar a direção garante que o pino assuma o valor pretendido no instante em que passa a ser saída. Na ordem inversa, o pino sai brevemente com o conteúdo anterior do latch — e, se ele aciona um relé ou um aquecedor, esse breve instante é um pulso real na carga.

5 Manipulação de bits

Operar sobre bits individuais sem afetar os vizinhos usa três operações, e vale memorizar o efeito de cada uma:

Objetivo	Operação	Exemplo
Ligar bits	OU com a máscara	<code>LATD = 0x81;</code>
Desligar bits	E com a máscara negada	<code>LATD &= (uint8_t)~0x81;</code>
Inverter bits	OU exclusivo com a máscara	<code>LATD ^= 0x81;</code>
Testar bits	E com a máscara	<code>if (PORTB & 0x01) { }</code>

Tabela 4: Operações de máscara.

Atenção

A conversão explícita na segunda linha não é preciosismo. Em C, o operando de 8 bits é promovido a inteiro antes da negação, e o resultado tem os bits altos ligados; sem a conversão de volta para 8 bits, o compilador emite um aviso legítimo. Ignorar avisos dessa natureza é como se aprende a conviver com defeitos reais escondidos no meio deles.

6 Ruído de contato

Definição --- ruído de contato

Sequência de transições espúrias que ocorre quando as lâminas metálicas de uma chave se tocam ou se separam, causada por micro-ressaltos mecânicos e pela elasticidade do material. Dura tipicamente de 1 ms a 20 ms, e produz de algumas a dezenas de transições.

Para o olho humano, o botão foi pressionado uma vez. Para um processador que executa uma instrução a cada 83 ns, houve uma rajada de eventos — e um contador incrementado a cada borda de descida registra 7, 12 ou 3 pressionamentos, de forma irreprodutível.

Observação

Note que o problema não é de software: o sinal elétrico **realmente** apresenta aquelas transições. Nenhum código as elimina; o que o código faz é decidir quais transições considerar. Filtragem, aqui, é uma decisão de projeto sobre o que constitui um evento.

6.1 Filtragem por espera

A abordagem direta: detectada a transição, esperar que o ruído termine e confirmar a leitura.

Código

```
if (PORTBbits.RB0 == 0) {           /* possível pressionamento */
    __delay_ms(20);                  /* espera o ruído cessar */
    if (PORTBbits.RB0 == 0) {        /* confirma */
        tratar_botao();
        while (PORTBbits.RB0 == 0) { } /* espera soltar */
    }
}
```

Funciona, é fácil de entender, e é a solução adequada para a primeira sessão de laboratório. Tem, porém, um defeito que se agrava com o projeto: durante os 20 ms de espera, **o programa não faz mais nada**. Não atualiza o display, não lê a temperatura, não responde à serial. E a última linha é pior: o programa fica detido enquanto o dedo permanecer no botão, por quanto tempo for.

Atenção

Este é o momento de reconhecer o padrão que reaparecerá em quase todos os encontros seguintes: **toda espera bloqueante é um empréstimo tomado contra o tempo de resposta do sistema**. Funciona enquanto o programa faz uma coisa só. Deixa de funcionar assim que passa a fazer duas.

6.2 Filtragem por amostragem periódica

A alternativa não bloqueante: em vez de esperar, amostrar o botão em intervalos regulares e exigir **estabilidade** — um número mínimo de leituras consecutivas iguais — antes de aceitar a mudança de estado.

Código

```
/* Chamada periodicamente, a cada 5 ms.
   Exige 3 leituras iguais para aceitar a mudança. */
uint8_t botao_amostrar(void)
{
    static uint8_t contador = 0;
    static uint8_t estado = 1;
    uint8_t leitura = PORTBbits.RB0;

    if (leitura == estado) {
        contador = 0;
        return 0; /* sem novidade */
    }

    if (++contador >= 3u) {
        contador = 0;
        estado = leitura;
        return (estado == 0); /* 1 se acabou de ser pressionado */
    }

    return 0;
}
```

Derivação

A largura do filtro é o produto do número de amostras exigidas pelo intervalo entre elas:

$$t_{\text{filtro}} = N \cdot T_{\text{amostra}}$$

Com $N = 3$ e $T_{\text{amostra}} = 5$ ms, o filtro rejeita perturbações de até 15 ms — suficiente para contatos comuns. Aumentar N melhora a imunidade e aumenta o atraso percebido; a partir de cerca de 100 ms, o usuário nota o botão “pesado”.

Observação

Esta função não espera: ela é chamada, decide e retorna. É a primeira aparição concreta do princípio que organiza todo o firmware do semestre — **uma tarefa que precisa de tempo não deve gastá-lo esperando, e sim ser chamada de novo mais tarde**. A base de tempo que fará essas chamadas regulares chega no encontro 7.

7 Corrente e limites elétricos

Derivação

Um LED em série com um resistor, acionado por um pino que fornece corrente:

$$R = \frac{V_{DD} - V_F}{I_F}$$

Para $V_{DD} = 5 \text{ V}$, LED vermelho com $V_F \approx 2,0 \text{ V}$ e corrente desejada de 10 mA :

$$R = \frac{5 - 2,0}{10 \cdot 10^{-3}} = 300 \Omega$$

O valor comercial imediatamente superior é 330Ω , que resulta em

$$I_F = \frac{5 - 2,0}{330} \approx 9,1 \text{ mA}$$

Escolher o valor superior, e não o inferior, é a prática correta: mantém a corrente abaixo do projetado.

Limite	Ordem de grandeza
Corrente por pino, drenada ou fornecida	25 mA
Corrente somada de um conjunto de portas	cerca de 200 mA
Corrente total do dispositivo	algumas centenas de mA

Tabela 5: Limites de corrente. Confirmar os valores na folha de dados do dispositivo antes de dimensionar.

Atenção

O limite por conjunto **não** é a soma dos limites individuais. Oito pinos a 25 mA dariam 200 mA, no limite exato do conjunto — sem margem alguma, e ignorando o consumo do próprio núcleo. Acionar oito LEDs a 20 mA cada é um projeto que funciona na bancada, aquece o chip e falha de forma intermitente. Cargas acima de poucos miliampères pedem transistor, e não pino.

Observação

Drenar ou fornecer. Um LED pode ser ligado entre o pino e o terra — o pino fornece a corrente, e nível alto acende — ou entre a alimentação e o pino, caso em que o pino drena a corrente e **nível baixo acende**. A segunda configuração é comum em placas didáticas e inverte a lógica do programa. Antes de concluir que o código está errado, vale verificar em que sentido o LED está ligado no kit.

8 Transposição

Em arquiteturas de 32 bits, a mesma estrutura reaparece com outros nomes: um registrador de direção, um de leitura das entradas e um de escrita nas saídas. As diferenças relevantes são três.

A primeira é a **configuração por pino**, e não por porta: modo de saída — par complementar ou apenas dreno aberto —, velocidade de comutação e resistor de elevação ou de abaixamento são escolhidos individualmente, ao contrário do resistor coletivo de uma única porta visto aqui.

A segunda é a existência de um **registrador de escrita atômica**, com metade dos bits dedicada a ligar e metade a desligar. Escrever nele altera apenas os bits pretendidos, em uma única operação — o problema da seção 3 deixa de existir por construção, e não por disciplina de programação.

A terceira é o **multiplexador de função alternativa**, que permite escolher, por software, qual periférico interno se conecta a cada pino. Boa parte dos conflitos de pinos que restringem o projeto nesta disciplina simplesmente não existiria ali.

Observação

O que não muda: os limites de corrente, a necessidade de transistor para cargas reais, o ruído de contato e a inconveniência das esperas bloqueantes. Essa é a divisão que o curso repete — **o que muda é a interface com o registrador; o que permanece é a física.**

9 Exercícios

Exercício 2.1

Um programa aciona um relé por RD3 e um LED por RD5, ambos por meio de operações do tipo `PORTDbits.RDn = 1;`. O relé passa a desarmar sozinho sempre que o LED é comutado. Explique o mecanismo do defeito e apresente a correção, justificando por que ele não aparecia quando somente o LED estava montado.

Exercício 2.2

Dimensione o resistor para um LED verde ($V_F = 2,2\text{ V}$) com corrente de 8 mA, alimentado em 5 V, e escolha o valor comercial adequado. Em seguida, calcule quantos LEDs iguais poderiam ser acionados simultaneamente pela mesma porta sem exceder o limite do conjunto, e comente se esse número é um projeto aceitável.

Exercício 2.3

Reescreva a função `botao_mostrar` da seção 6 para tratar **dois** botões independentes, mantendo-a não bloqueante e sem duplicar a lógica de filtragem. Indique quais variáveis precisam deixar de ser escalares.

Exercício 2.4

Um estudante configura `TRISC = 0x00` e escreve `LATC = 0xFF`, mas mede 0 V em dois dos oito pinos, enquanto os demais apresentam 5 V. O chip é novo e o código não tem outros erros. Explique o que ocorre e diga se há configuração capaz de corrigir.

Exercício 2.5

Compare as duas estratégias de filtragem de contato quanto a: tempo de resposta percebido pelo usuário, imunidade a ruído, impacto sobre outras tarefas do sistema e facilidade de depuração. Em seguida, defenda a escolha da espera bloqueante em um cenário específico — ela existe — e explique por que esse cenário desaparece a partir do encontro 7.